

# F E I

## Tentamen i Programmeringsteknik 2000-05-24, kl 8.00-13.00

### *Anvisningar*

Hjälpmedel: Holm, Objektorienterad programmering och Java, samt Java-snabbreferens.

Skrivningen innehåller fem uppgifter. Du bör behandla samtliga uppgifter.

Obs. Senast 3:e arbetsdagen efter tentamen anslås på anslagstavlan vilka som deltagit i tentamen men som, enligt institutionens noteringar, inte uppfyller tentamensvillkoren (alla 18 övningarna och de 3 inlämningsuppgifterna godkända). Deras skrivningar annulleras den 6:e arbetsdagen efter tentamen, såvida inte rättelse skett innan dess eller ansvarig lärare lämnat dispens.

Närmaste tentamenstillfällen i kursen:

Fredag 25 augusti 2000, kl 8-13

Lördag 13 januari 2001, kl 14-19

Måndag 5 mars 2001, kl 8-13

### *Något om bedömningen*

- Tänk på att du med dina lösningar vänder dig till en mänsklig rättare, inte till en dator.
- Du kan förutsätta att rättaren är väl insatt i uppgifterna.
- Rättaren fäster ingen större uppmärksamhet vid fel som en kompilator enkelt skulle upptäcka.
- Det är viktigt att du gör lämpliga och tydliga indragningar och att du använder vettiga variabelnamn så att din lösning går lätt att följa. En god handstil är ett plus.
- Tvekar du mellan att presentera en smart lösning eller en lösning som är enkel att följa, välj den enklare versionen.
- När det gäller betyget 3 så har det ingen betydelse om någon lösning är onödigt ineffektiv.

## Problemet

I tentamensskrivningen ingår fem uppgifter, alla har samma bakgrund: Broloppet den 12 juni 2000 från Köpenhamn till Malmö.

Vid broloppet skall man hålla reda på löparna och deras tider vid olika kontroller (inkl målet). Eftersom så många löpare är anmälda, drygt 90 000, startar löparna i grupper om ca 1 500 med några minuters mellanrum mellan grupperna.

Varje löpare är utrustad med en elektronisk "fotboja" som sänder en signal. Vid varje kontroll uppfångas signalen och information om vilken löpare som passerat kontrollen och tidpunkten "clock" för passagen sänds till en central dator. Genom att från "clock" subtrahera löparens "startTime" kan man räkna ut hans mellantid "cTime" vid den aktuella kontrollen. Tiderna räknas i sekunder (heltal). Tidpunkten för de första löparnas "startTime" är  $9 \cdot 3600$  (=kl 9.00).

Du skall skriva delar av ett programsystem som ger "allmänheten" information om löparnas tider och placeringar vid olika kontroller. Följande utskrifter skall kunna begäras:

- aktuell ställning vid en given kontroll: löparna i ordning från den med minsta mellantiden (cTime) till den med största mellantiden. Ingen speciell hänsyn behöver tas till att flera löpare kan ha samma mellantid.
- var en given löpare befinner sig för tillfället dvs besked om den kontroll han senast passerade tillsammans med den tidpunkt (clock) när detta skedde (används t ex vid WAP-telefoni).

Du skall skriva fyra klasser:

- Runner, som håller reda på informationen om en löpare: nummer, namn, starttid, senast passerade kontroll.
- SplitTime, som håller reda på mellantiden för en löpare vid en kontroll. Ett SplitTime-objekt innehåller en referens till löparen och hans mellantid (cTime).
- Control, som beskriver en kontroll med dess avstånd från start och som innehåller en vektor med SplitTime-objekt för att hålla reda på mellantiderna (cTime) för de löpare som passerat kontrollen. Vektorn hålls sorterad med avseende på mellantiderna.
- Organizer, som håller reda på alla löpare och kontroller och som innehåller operationer som ser till att allmänheten kan få de ovan önskade utskrifterna.

## Uppgift 1

Klassen Runner har nedanstående uppbyggnad. Implementera konstruktorn och de fem operationerna i klassen!

```
class Runner {
    private int nbr;           // löparens nummer
    private String name;       // löparens namn
    private int startTime;     // löparens starttid (vad klockan visade)
    private Control lastControl; // senast passerade kontroll

    /** skapa en löpare */
    public Runner(int nbr, String name, int startTime) {
        ... // implementera!
    }

    /** tag reda på löparens nummer */
    public int getNbr() {
        ... // implementera!
    }

    /** tag reda på löparens namn */
    public String getName() {
        ... // implementera!
    }

    /** tag reda på löparens starttid */
    public int getStartTime() {
        ... // implementera!
    }

    /** notera senast passerade kontroll */
    public void setLastControl(Control control) {
        ... // implementera!
    }

    /** tag reda på senast passerade kontroll */
    public Control getLastControl() {
        ... // implementera!
    }
}
```

## Uppgift 2

Ett objekt av klassen SplitTime innehåller en referens till en löpare och håller reda på hans mellantid (uttryckt i sekunder). Parametern cTime i konstruktorn är alltså hans mellantid, inte tidpunkten när löparen passerade kontrollen. I varje kontroll skall löparna sedan hållas sorterade med avseende på mellantiderna (sorteringen görs i uppgift 3).

Klassen SplitTime har nedanstående uppbyggnad. Implementera konstruktorn och de två operationerna!

```
class SplitTime {
    private Runner runner;     // löparen
    private int cTime;         // mellantiden

    /** skapa ett objekt som beskriver en löpare med mellantiden cTime */
    public SplitTime(Runner runner, int cTime) {
        ... // implementera!
    }

    /** tag reda på löparen */
    public Runner getRunner() {
        ... // implementera!
    }

    /** tag reda på löparens mellantid */
    public int getCTime() {
        ... // implementera!
    }
}
```

## Uppgift 3

Ett objekt av klassen `Control` beskriver en kontroll med dess avstånd från start och innehåller en vektor med mellantider för löparna (`SplitTime`-objekt). Vektorn innehåller de löpare som passerat kontrollen och är sorterad efter löparnas mellantider vid kontrollen.

Klassen `Control` har nedanstående uppbyggnad. Implementera konstruktorn och de fem operationerna!

```
class Control {
    private double distance;          // kontrollens avstånd från start
    private SplitTime[] splitVec;     // vektor med de löpare som passerat kontrollen
    private int nbrPassed;            // antal löpare som passerat kontrollen

    /** skapa en kontroll, maxNbrPassed är övre gräns för antal passerande löpare */
    public Control(double dist, int maxNbrPassed) {
        ... // implementera!
    }

    /** tag reda på kontrollens avstånd från start */
    public double getDistance() {
        ... // implementera!
    }

    /** notera en ny löpare som passerar kontrollen vid tidpunkten clock,
        räkna ut hans mellantid och sortera in honom i vektorn */
    public void runnerPassed(Runner runner, int clock) {
        ... // implementera!
    }

    /** tag reda på hur många som passerat kontrollen */
    public int getNbrPassed() {
        ... // implementera!
    }

    /** tag reda på SplitTime-objektet på viss plats (1:a, 2:a, etc) i vektorn */
    public SplitTime getSplitTime(int place) {
        ... // implementera!
    }

    /** tag reda på tidpunkten när löparen runner passerade kontrollen,
        om runner ännu inte passerat erhålls resultatet -1 */
    public int getClock(Runner runner) {
        ... // implementera!
    }
}
```

## Uppgift 4

Klassen Organizer, som håller reda på alla löpare och kontroller och har operationer som ser till att allmänheten kan få önskade resultatutskrifter, har nedanstående uppbyggnad. Skriv de två operationerna reportStanding och reportRunner.

```
class Organizer {
    private Runner[] runnerVec;        // vektor med alla anmälda löpare i broloppet
                                        // finns i elementen 1..nbrOfParticipants
                                        // löparens nummer är samma som hans plats i vektorn

    private int nbrOfParticipants;      // totalt antal anmälda löpare
    private Control[] controlVec;       // vektor med kontrollerna
                                        // finns i elementen 0..nbrOfControls-1
    private int nbrOfControls;          // antal kontroller

    /** konstruktor */
    public Organizer(...) {
        // denna konstruktor skall du inte skriva!
    }

    /** notera en ny anmäld löpare */
    public void addRunner(Runner runner) {
        // denna operation skall du inte skriva!
    }

    /** notera en ny kontroll */
    public void addControl(Control control) {
        // denna operation skall du inte skriva!
    }

    /** skriv ut den aktuella ställningen vid kontrollen control,
        utskriften skall ske på enheten writer (fungerar som en ConsoleWriter)
        och skall ha följande innehåll:

        Vid kontrollen <avståndet från start> km är ställningen just nu:
        1. <löparens nummer, namn> <löparens mellantid i timma, minut, sekund>
        2. <löparens nummer, namn> <löparens mellantid i timma, minut, sekund>
        ...

        Obs. Du behöver inte vara exakt med formateringen av utskriften,
        det viktiga är att rätt text och resultat skrivs ut i rätt ordning.
    */
    public void reportStanding(Control control, Writer writer) {
        ... // implementera!
    }

    /** skriv ut vilken kontroll som löparen med nummer nbr senast passerade och
        tidpunkten när detta skedde, eller om han ännu ej har passerat någon kontroll,
        skriv ut detta. Utskriften kan se ut enligt följande:

        <löparens nummer, namn> passerade kontrollen <kontrollens avstånd> klockan
        <här bara timma, minut>
        eller
        <löparens nummer, namn> har ännu ej passerat någon kontroll
    */
    public void reportRunner(int nbr, Writer writer) {
        ... // implementera!
    }
}
```

## Uppgift 5

I uppgift 3 används i klassen Control en vektor splitVec av SplitTime-objekt för de löpare som passerat den aktuella kontrollen. Alternativt kan man tänka sig att SplitTime-objekten lagras i en länkad lista med hjälp av jlist-paketet:

```
class SplitTime extends Link {
    // samma som SplitTime i uppgift 2
}
```

Klassen Control skulle då kunna få nedanstående uppbyggnad. Implementera kon-  
struktor och de fyra ändrade operationerna i den nya Control-klassen!

```
class Control {
    private double distance; // kontrollens avstånd från start
    private Head list;      // lista med de löpare som passerat kontrollen

    /** skapa en kontroll */
    public Control(double dist) {
        ... // implementera!
    }

    /** tag reda på kontrollens avstånd från start */
    public double getDistance() {
        ...
    }

    /** notera en ny löpare som passerar kontrollen vid tidpunkten clock,
        räkna ut hans mellantid och sortera in honom i listan */
    public void runnerPassed(Runner runner, int clock) {
        ... // implementera!
    }

    /** tag reda på hur många som passerat kontrollen */
    public int getNbrPassed() {
        ... // implementera!
    }

    /** tag reda på SplitTime-objektet på viss plats (1:a, 2:a, etc) i listan */
    public SplitTime getSplitTime(int place) {
        ... // implementera!
    }

    /** tag reda på tidpunkten när löparen runner passerade kontrollen,
        om runner ännu inte passerat erhålls resultatet -1 */
    public int getClock(Runner runner) {
        ... // implementera!
    }
}
```